CSC3150 Operating Systems · Midterm Cheat Sheet (Side A: Concepts, Syscalls, Files, IPC & Synchronization)

Nov 5, 2026 10:30–12:00 @ AB E100 · 1 Sheet Double-Sided

1. OS 角色、评价指标与四大基本概念 (L1–L2)

▸ **OS 三大角色：(1) Referee（裁判/资源隔离）**：进程隔离、双模式保护、CPU 调度、IPC 访问控制；**(2) Illusionist（魔术师/硬件虚拟化）**：时间分片虚拟 CPU（`cpu.c` 靠硬件定时器中断抢占；无抢占协同式系统如 Win3.1/MacOS9 死循环会卡死全机）、独立虚拟地址空间（`memory.c` 关 ASLR 后两进程打印相同虚拟地址如 `0x200000`，但经页表映射到不同物理内存）；**(3) Glue（胶水/公共服务）**：文件系统、网络协议栈、设备驱动、GUI/Shell。

▸ **六大评价指标：** 开销（Overhead）、公平性（Fairness）、可移植性（Portability，HAL 屏蔽硬件）、可靠性（Reliability：$\text{Availability} = \frac{\text{MTTF}}{\text{MTTF}+\text{MTTR}}$）、安全性（Security：完整性 Integrity + 机密性 Privacy）、性能（Performance：**响应时间 Response Time** 单任务耗时、**吞吐量 Throughput** 单位时间完成任务数、**可预测性 Predictability** 方差小/实时保证）。

▸ **概念 1：线程 (Thread)**：CPU 调度的基本单位 = **单个独立执行上下文 (Execution Context)**：包含 **PC（程序计数器）、通用寄存器集合、标志位（Flags）、执行栈（Stack，SP）**，保存在 **TCB** 中。Running = 上下文在 CPU 寄存器中；Suspended = 上下文保存在内存 TCB 中。

▸ **概念 2：地址空间 (Address Space)**：从低地址到高地址四段：**(1) Code/Text**（只读机器指令）→ **(2) Static Data**（全局变量、`static` 变量）→ **(3) Heap**（`malloc/new` 动态分配，**向高地址增长**）→ **(4) Stack**（局部变量、参数、返回地址，**随调用向低地址增长**）。注意：`int *p = malloc(4)` 中指针变量 `p` 本身在**栈**上，`*p` 指向的 4 字节块在**堆**上！

▸ **概念 3：进程 (Process)**：资源分配与保护隔离的单位 = **1 个受保护的虚拟地址空间 + 1 个或多个线程 + 文件描述符表等内核资源**（保存在 PCB 中）。

- **正交分解**：线程封装**并发性**（主动执行流），地址空间封装**保护与隔离**（被动边界）。
- **进程 vs 线程权衡【Sample Q1a】**：同进程多线程共享代码、全局数据、堆、fd 表，**优点**是创建/切换极快（不换页表/TLB）、共享内存通信零拷贝；**缺点**是无内存隔离，单线程野指针越界或段错误会导致整个进程崩溃。

▸ **概念 4：双模式运行 (Dual Mode)【Sample Q1b】**：硬件提供模式位区分 **User Mode (`sysmode=0`)** 与 **Kernel Mode (`sysmode=1`)**。

- **目的**：保护 OS 与其他进程免受缺陷或恶意用户程序破坏。
- **用户态禁止的 4 类特权操作**：(1) 修改地址转换寄存器（Base/Bound 或页表基址）；(2) 关/开硬件中断；(3) 直接访问物理 I/O 设备端口；(4) 访问内核专用内存。违例触发硬件异常陷入内核。
- **Base & Bound 保护**：版本 1（加载时重定位，运行时只比大小 $\text{Base} \le \text{addr} < \text{Bound}$）；版本 2（运行时硬件转换：$\text{PhysAddr} = \text{Base} + \text{VirtAddr} < \text{Bound}$）。内核用特权指令 `RTU`（Return To Usermode）切回用户态。

▸ **进入内核的三种非编程控制转移：** 均通过硬件查阅内核预设的**中断向量表 (Interrupt Vector Table)** 跳转到受控入口 `intrpHandler_i()`（用户绝不能自己指定跳转内核地址！）：

1. **系统调用 (Syscall)**：**同步、主动**（用户执行 `syscall/ecall` 请求 `read/fork` 等）。
2. **中断 (Interrupt)**：**异步、外部硬件触发**（定时器到期、磁盘/网卡 I/O 完成，与当前指令无关）。
3. **陷阱/异常 (Trap/Exception)**：**同步、当前指令内部触发**（除以 0、非法指令、缺页/越界违例）。

2. 上下文切换、pthread 与栈帧追踪 (L3 & Sample Q1d)

▸ **定时器中断切换进程四步流程：** (1) P1 在用户态运行；(2) 定时器中断到达 → 硬件自动置 `sysmode=1`、保存用户态 `uPC`、跳入 `IntrpVector[i]`；(3) 内核处理程序把 P1 通用寄存器与 `uPC` 存入 P1 的 PCB，切内核栈，调 Scheduler 选就绪进程 P2，更新 `Base/Bound` 为 P2，从 P2 的 PCB 恢复寄存器与 `uPC`；(4) 执行 `RTU` 返回用户态运行 P2。

▸ **并发术语辨析：** **Multiprocessing** = 多物理 CPU 核；**Multiprogramming** = 多作业驻留交替执行；**Multithreading** = 单进程内多线程；**Concurrency（并发）** = 同一时间段内交叠**处理（dealwith）**多任务（单核分时即可）；**Parallelism（并行）** = 同一物理时刻在多核上同时**执行（do）**多任务。

▸ **Pthreads API 与指针设计必考题【Sample Q1d 原题！】：**

```
int pthread_create(pthread_t *thread, const pthread_attr_t *attr,
                   void *(*start_routine)(void *), void *arg);
void pthread_exit(void *value_ptr);
int pthread_join(pthread_t thread, void **value_ptr);
```

- **为什么 `thread` 是指针 `pthread_t *`？** C 语言按值传参；`pthread_create` 需要在函数内部生成新线程 ID 并**回写（output parameter）**给调用方变量，必须传变量地址才能通过 `*thread = new_tid` 修改。
- **为什么 `arg` 是 `void *`？** 为了向线程入口函数传递**任意类型、任意大小**的参数（传结构体地址或强转的整数），`void *` 是 C 泛型指针。
- **为什么 `pthread_exit` 的 `value_ptr` 是 `void *`？** 允许线程返回**任意类型**的结果指针（注意：决不能返回指向本线程局部栈变量的指针，因线程退出后栈帧销毁！）。
- **为什么 `pthread_join` 的 `value_ptr` 是二级指针 `void **`？** 因为被回收的返回值本身是一个 `void *` 指针；`pthread_join` 要把这根 `void *` 指针**回写**给调用方声明的 `void *ret` 变量，必须传入 `ret` 的地址 `&ret`（即 `void **`），内部执行 `*value_ptr = exited_value_ptr`。
- **注意**：工作线程 `return val` 等价于 `pthread_exit(val)`；但 **`main` 函数执行 `return` 会直接调用 `exit()` 杀死整个进程及所有线程**（除非 `main` 调 `pthread_exit`）！

▸ **向线程传栈局部变量地址的 Bug (L7)：** 主线程循环 `int fd = accept(...); pthread_create(&t, NULL, worker, &fd);` 会产生数据竞争：下一轮循环修改 `fd` 时，子线程解引用 `&fd` 会读到被覆盖的新连接描述符！**修复**：`int *p = malloc(sizeof(int)); *p = fd;` 传 `p`，或将整数强转为 `(void *)(intptr_t)fd` 按值传递。

3. 进程 API、文件描述符与 IPC (L4–L6 & Sample Q2/Q7.1)

▸ **`fork()` 精确语义【Sample Q2.1 原题！】：**

- **返回值**：**父进程得子进程 PID（> 0）；子进程得 `0`**；失败返回 `-1`。
- **逻辑复制的内存**：复制父进程整个虚拟地址空间（代码、全局数据、堆、栈）及文件描述符表（现代 OS 用 **Copy-on-Write 写时复制**：先共享只读物理页，任一方写入时触发缺页才真正拷贝物理页）。**注意：子进程只复制调用 `fork()` 的那一个线程！**连续无条件调用 k 次 `fork()` 共产生 2^k 个进程。
- **共享 vs 不共享**：**共享**内核打开文件描述（Open File Description，含**当前文件读写偏移量 `offset`**）；**不共享** PID/PPID、虚拟内存修改、待处理信号（Pending signals）、文件锁。

▸ **`exec`、`wait` 与信号：**

- `execv(path, argv)`：用新程序替换当前进程的代码/数据/堆/栈（**PID 不变**，打开的 `fd` 默认保持打开）；**一旦成功永不返回**，只有失败才返回 `-1`。
- **为什么拆成 `fork` + `exec`？** 允许在 `fork()` 之后、`exec()` 之前在子进程里用普通代码修改 `fd`（如 `close/dup2` 实现 Shell 重定向 `>` 和管道 `|`）。
- **信号**：`SIGINT`（Ctrl–C）、`SIGTERM` 默认终止；`SIGTSTP`（Ctrl–Z）暂停；**`SIGKILL` (9) 与 `SIGSTOP` 不能被 `sigaction` 捕获、阻塞或忽略**！

▸ **高层 `FILE*` vs 底层 `fd`【Sample Q7.1 原题！】：**

维度	高层 C 流（`FILE*`，`stdio.h`）	底层系统调用（`int fd`，POSIX）
接口与位置	`fopen/fread/fwrite/fprintf/fgets`，**用户态结构体含缓冲区**	`open/read/write/lseek/close`，**进程 fd 表整数下标** (0/1/2)
调用开销与可见性	先写用户态缓冲，满/遇 `\n`/调 `fflush` 才 syscall；`printf("A ");sleep(10);` 会等 10s 后才随 `\n` 一起印出	每次调用立即陷入内核；`write(1,"A ",2);sleep(10);` 立即印出 A
各自优势	**减少高昂的系统调用次数**；提供格式化与按行读取 (`fgets`)	**无缓冲延迟、精确控制字节数与偏移**；可用于套接字/管道/设备非阻塞控制

- **`read(fd, buf, max)` 返回值**：`>0` 实际读到字节数（可少于 `max`，叫 short count）；**`0` 表示 EOF**；**`-1` 表示错误**（设 `errno`）。
- **内核两级文件表**：进程私有 `fd` 表 → 系统全局**打开文件描述表（`struct file`，存磁盘 inode 指针、当前 `offset`、引用计数 `refcnt`）**。
  - 两人各自 `open("a.txt")` → **两个独立** `struct file`，各自独立 `offset`。
  - 先 `open` 得 `fd=3`（读 100B 使 `offset=100`）再 `fork()` → 父子 `fd=3` 指向**同一个** `struct file`（`refcnt=2`），**共享 `offset`**！父再读 100B 后，子接着读会从 `200..299` 开始读！父调 `close(3)` 仅使 `refcnt` 减为 1，子仍可读。

▸ **Sockets 与服务器演进 (L6)：**

- **TCP 五元组**：（源 IP，目的 IP，源端口，目的端口，TCP）唯一标识一条连接（故同一 80 端口可并发服务上万客户端）。端口 `0–1023` 需 root 权限。
- **调用序列**：Server `socket -> bind -> listen -> conn_fd = accept(listen_fd)`（返回全新连接 fd！）；Client `socket -> connect`。
- **五版本服务器**：v1 单进程串行；v2 `fork`+父立即 `wait`（有故障隔离但仍串行！）；v3 `fork` 不阻塞 `wait`（多进程并发，**父进程必须 `close(conn_fd)`**，否则 `refcnt` 不归零导致连接永不发 FIN 关闭且耗尽父进程 fd；子进程关闭 `listen_fd`）；v4 每连接 `pthread_create`（开销小但无内存隔离）；v5 **有界线程池**（防突发高并发撑爆内存与切换开销）。

▸ **匿名管道 `pipe(fd)` 与 IPC 选型【Sample Q2.2 原题！】：**

- `pipe(fd)`：**`fd[1]` 写端，`fd[0]` 读端**。只有当**所有指向写端 `fd[1]` 的描述符全部关闭（写端 `refcnt==0`）**时，读者 `read(fd[0])` 才会返回 `0`（EOF）！故 `fork` 后子进程读者必须先 `close(fd[1])`，否则永远阻塞等自己。
- **无亲缘关系的同机两进程通信选型【Sample Q2.2】**：
  - **匿名 Pipe**：**不合适**！无全局名字，只能靠共同祖先 `fork()` 继承 `fd`。
  - **Socket（Unix Domain Socket / Loopback TCP）**：**合适**！通过路径名或 `127.0.0.1:port` 建立双向字节流通信。
  - **Shared Memory（`shm_open/mmap`）**：**合适**！通过全局名字映射同一块物理内存，零拷贝最快，但**必须自行配合互斥锁/信号量同步**。

4. 原子操作、锁实现、信号量与管程 (L7–L9 & Sample Q3)

▸ **竞争条件与软件互斥 (L7)：**

- `counter++` 编译为 `load r1, M[c]; add r1, 1; store r1, M[c]` 三条指令，交错导致更新丢失。现代 CPU 上**对齐的单字（word）`load/store` 是原子的**。
- **Too Much Milk**：方案 1（先查 `no_note` 再贴 `note=1`）违反**安全性**（可两人同时买）；方案 2（A 贴 `noteA=1` 后若 `noteB==0` 则买，B 对称）违反**活性**（两人同时贴条后都放弃买，死锁式饥饿）；方案 3（A `while(noteB)` 忙等，B `if(!noteA)`）正确但代码不对称、忙等、依赖顺序一致性。
- **Peterson 双线程互斥算法**（`i` 为本线程，`j = 1 - i` 为对方）：`flag[i] = true; turn = j; while (flag[j] && turn == j); /* CS */ flag[i] = false;`

▸ **互斥锁的四种实现机制 (L8)：**

1. **单核关中断锁**：不能给用户态用；**多核完全失效**（关本核中断管不住别核访存）。不能在整个临界区关中断，应只在修改锁变量和等待队列时关中断，且**必须在保持关中断下调 `sleep()` 进入调度器**（若在入队后、`sleep` 前开中断，持锁者此时 `Release` 会导致**丢失唤醒 Lost Wakeup**！由切换后的下一线程负责开中断）。
2. **`test&set` 自旋锁**：`while (test_and_set(&lock) == 1);` 浪费 CPU，且每次原子写造成多核缓存行反复失效（Cache ping-ponging）。**Lec13 改进 `test&test&set`**：`do { while(lock == 1); } while(test_and_set(&lock));` 先在本地缓存纯读等待（零总线流量），看到变 0 才发起原子写！
3. **`guard` 自旋 + 睡眠队列锁**：用 `while(test_and_set(&guard));` 仅保护对锁状态 `value` 和等待队列的极短操作；若 `value == BUSY` 则入队睡眠并清零 `guard`。
4. **Linux `futex(uaddr, op, val)`**：无竞争时纯用户态 `test_and_set` 一条指令拿锁（零 syscall）；有竞争时调 `futex(addr, FUTEX_WAIT, 1)`，内核**原子检查 `*addr == 1` 仍成立才入睡**，彻底消除检查与睡眠之间的丢失唤醒！

▸ **信号量 (Semaphore) 与有界缓冲区【Sample Q3 16 分原题！】：**非负整数 s：`wait(s)` / `P(s)` 当 $s > 0$ 时减 1，若 $s == 0$ 则阻塞；`signal(s)` / `V(s)` 将 s 加 1 并唤醒等待者。初值 `1` 作互斥锁，初值 `0` 作事件顺序同步。

```
// Shared: Item buf[N]; int head=0, tail=0; semaphore empty=N,
full=0; mutex m;
void put(Item x) {                  Item get() {
    wait(empty); // 1. MUST first!     wait(full);  // 1. MUST first!
    lock(m);     // 2. Then mutex      lock(m);     // 2. Then mutex
    buf[tail] = x;                     Item x = buf[head];
    tail = (tail + 1) % N;             head = (head + 1) % N;
    unlock(m);                         unlock(m);
    signal(full);                      signal(empty);
}                                      return x;
                                   }
```

- **Q3.2 删掉 `mutex m` 只留 `empty/full` 行吗？绝对不行（Wrong）！** 当 $N > 1$ 且有多个生产者（或消费者）时，两个生产者 P1、P2 可同时通过 `wait(empty)` 进入临界区，同时读取相同的 `tail` 下标，把各自的数据写进同一个槽 `buf[tail]` 造成**数据覆盖丢失**，且 `tail` 更新竞争！
- **Q3.3 调换 `wait(empty)` 和 `lock(m)` 顺序会怎样？会导致死锁（Deadlock）！** 当缓冲区已满（`empty == 0`）时，生产者先 `lock(m)` 拿走互斥锁，再调 `wait(empty)` 带着锁睡眠；此时消费者想调 `get()` 取走元素腾出空位，却在 `lock(m)` 处被睡眠的生产者永久阻塞，双方循环等待死锁！
- **Q3.4 代码只有 `wait(empty); buf[tail]=x; tail=(tail+1)%N; signal(full);` 缺什么？** 缺少互斥锁 `lock(m)/unlock(m)` 保护共享数组与 `tail` 下标更新，多生产者并发时产生 Race Condition（覆盖同一槽位、丢失元素）。

▸ **管程 (Monitor)、Mesa 语义与读写锁 (L9)：**

- **管程** = 1 把互斥锁 + 条件变量 `cv`（**`cv` 无值记忆**，无人等时 `signal` 是空操作 No-op！）。`cond_wait(&cv, &lock)` 三步：**原子释放 `lock` 并睡眠** → 被唤醒移入就绪队列 → **返回前重新获取 `lock`**。
- **Hoare vs Mesa 语义**：Hoare 下 `signal` 立即把锁和 CPU 移交给被唤醒者（可用 `if`）；**Mesa 语义（真实 OS）**下发信号者继续运行，被唤醒者只进就绪队列，等它再抢到锁时状态可能已被别的线程改掉，故**条件检查必须用 `while (!cond) cond_wait(&cv, &lock);`**！
- **写者优先读者–写者锁**（计数器 `AR, WR, AW, WW` 初值 0）：
  - Reader：`acquire(&lock); while (AW + WW > 0) { WR++; cond_wait(&ok_to_read, &lock); WR--; } AR++; release(&lock);` 读库后：`acquire(&lock); AR--; if (AR == 0 && WW > 0) cond_signal(&ok_to_write); release(&lock);`
  - Writer：`acquire(&lock); while (AW + AR > 0) { WW++; cond_wait(&ok_to_write, &lock); WW--; } AW++; release(&lock);` 写库后：`acquire(&lock); AW--; if (WW > 0) cond_signal(&ok_to_write); else if (WR > 0) cond_broadcast(&ok_to_read); release(&lock);`

CSC3150 Operating Systems · Midterm Cheat Sheet (Side B: AI Systems, CPU Scheduling, EEVDF & Deadlocks)

Lectures 10–14 + Sample Exam Q4 (MLFQ) & Q5 (Deadlock)

5. 现代 AI 系统中的并发与集群调度 (L10 & L14)

- **并发缺陷基准 (L10)**：**RaceBench (CCS'23)**：构造可触发、可观测的并发 bug 基准；**Go 并发 Bug 研究 (ASPLOS'19)**：分为阻塞型（死锁）与非阻塞型（数据竞争），使用消息传递 `channel` 引起的并发 bug 并不比共享内存锁少。
- **DNN 训练三种并行范式与 PipeDream (SOSP'19, L10)**：
- **Data Parallelism (数据并行)**：每 Worker 存完整模型副本、跑不同数据分片，每步 `AllReduce` 同步全量梯度 $\nabla W = \sum \nabla W_i$，大模型下**通信开销极大**。
- **Model Parallelism (朴素模型并行)**：按层切分到不同 Worker；因单 minibatch 前后向严格依赖，**同一时刻仅 1 个 Worker 计算，硬件利用率极低**。
- **PipeDream 流水线并行**：连续注入多个 minibatch，三大核心机制：
  1. **阶段划分与按需复制**：使各阶段计算时间 t_k 均衡，且切分边界处激活张量通信量 t^{comm} 最小，慢阶段可做数据并行复制（如 `2-3-2-1`）。
  2. **1F1B (One–Forward–One–Backward) 调度**：稳定期每 Worker 严格交替执行 1 次前向与 1 次反向，消除流水线气泡并及时更新参数。
  3. **Weight Stashing (权重版本暂存)**：第 n 个 minibatch 前向用的是旧权重 $W^{(n)}$，等它反向传回该阶段时权重已被后续批次更新为 $W^{(n+p)}$；**必须暂存 $W^{(n)}$ 供第 n 批反向计算梯度**，保证同一批次前反向权重版本严格一致！
- **LLM 推理服务系统：Orca 与 DistServe (L10)**：
- **Orca (OSDI'22)**：传统请求级批处理 (Request-level batching) 必须等整批中最长回答生成完才能释放已完成请求的显存。Orca 提出 **(1) 迭代级调度 (Iteration-level Scheduling / Continuous Batching)**：每生成 1 个 token 的单次迭代后立即移出已结束请求并插入新请求；**(2) 选择性批处理 (Selective Batching)**：仅对依赖序列长度的 Attention 按请求分开算，对线性层矩阵乘法将所有 token 拼成二维张量统一批处理。
- **DistServe (OSDI'24)**：将 LLM 推理解耦为两个特征截然不同的阶段：

阶段	计算特征与硬件瓶颈	决定的延迟 SLO 指标
Prefill（预填充）	并行处理全部输入 Prompt tokens 生成首 token；**计算密集型 (Compute-bound)**	**TTFT** (Time To First Token)
Decode（解码）	每步自回归生成 1 个 token，需读全部历史 KV Cache；**访存带宽密集型 (Memory-bound)**	**TPOT** (Time Per Output Token)

  共置 (Colocation) 会导致长 Prefill 阻塞同批 Decode (拉高 TPOT) 且并行策略耦合。**DistServe 将 Prefill 与 Decode 分离到不同 GPU 集群**，独立配置最优张量并行 (TP) /流水线并行 (PP) 与实例配比，并以高带宽机内/机间网络传输 KV Cache，最大化单卡 **Goodput** (满足 TTFT/TPOT SLO 的最大吞吐)。
- **现代集群调度三篇论文 (L14)**：
- **Firmament (OSDI'16)**：将大规模数据中心任务放置建模为图上的**最小费用最大流 (Min-Cost Max-Flow)**，利用集群状态缓变特征运行**增量式流求解**，在万台节点上兼顾复杂亲和性约束与亚秒级 (Sub-second) 调度延迟。
- **Gandiva (OSDI'18)**：利用深度学习训练 **mini-batch 迭代周期高度可预测、且在迭代边界处中间激活张量已释放 (显存占用处于波谷)** 的特征，在迭代边界执行低开销**时间片抢占切换 (Time-slicing)、跨节点动态迁移与多任务单卡打包 (Packing)**。
- **Synergy (OSDI'22)**：打破"每张 GPU 固定绑定成比例 CPU 核与内存 (GPU-proportional)"的限制，在线剖析不同 DNN 作业数据预处理对 CPU/内存的**敏感度曲线 (Sensitivity profiling)**，跨作业异构配比 CPU/内存以消除 GPU 数据饥饿 (Data stall)。

6. CPU 调度经典策略与甘特图算例 (L11)

- **基本公式与目标冲突**：$\text{Turnaround} = T_{\text{completion}} - T_{\text{arrival}}$；无 I/O 时 $\text{Waiting} = T_{\text{turnaround}} - T_{\text{CPU}}$；有 I/O 时 $\text{Waiting} = T_{\text{turnaround}} - T_{\text{CPU}} - T_{\text{IO}}$；$\text{Response} = T_{\text{first_run}} - T_{\text{arrival}}$。
- **三大目标权衡**：最小化响应时间 (需频繁抢占) vs 最大化吞吐量 (需减少上下文切换开销并保持 CPU+I/O 设备同时忙碌) vs 公平性 (**Slide 6：Fairness ≠ 最小化平均响应时间**！优先跑短作业虽不公平但平均响应时间更低)。

1. FCFS / FIFO（非抢占）与护航效应 (Convoy Effect)

- **Lec11 Slide 8–9 算例**：$P_1 = 24, P_2 = 3, P_3 = 3$ 在时刻 0 到达。
  - 顺序 P_1, P_2, P_3：等待 $0, 24, 27 \Rightarrow \text{Avg Wait} = \mathbf{17}$，完成 $24, 27, 30 \Rightarrow \text{Avg Comp} = \mathbf{27}$（短任务卡在长任务后，即 **Convoy Effect**）。
  - 顺序 P_2, P_3, P_1：等待 $6, 0, 3 \Rightarrow \text{Avg Wait} = \mathbf{3}$，完成 $30, 3, 6 \Rightarrow \text{Avg Comp} = \mathbf{13}$。

2. Round Robin (RR，时间片轮转)

- **Lec11 Slide 13 算例 ($q = 20$)**：$P_1 = 53, P_2 = 8, P_3 = 68, P_4 = 24$ 在时刻 0 到达。甘特图：`P1(0–20) -> P2(20–28,完) -> P3(28–48) -> P4(48–68) -> P1(68–88) -> P3(88–108) -> P4(108–112,完) -> P1(112–125,完) -> P3(125–153,完)`。完成：$P_1 = 125, P_2 = 28, P_3 = 153, P_4 = 112$（平均 **104.25**）；等待：$P_1 = 72, P_2 = 20, P_3 = 85, P_4 = 88$（平均 **66.25**）。
- **时间片 q 经验值 (Slide 16)**：$q \to \infty$ 退化为 FCFS；q 太小则切换开销占比过高吞吐崩溃。现代 OS 上下文切换约 $\mathbf{0.1–1\ ms}$，取 $\boldsymbol{q} \in [\mathbf{10\ ms, 100\ ms}]$（使切换开销占比 $\approx 1\%$）。
- **等长作业下 RR 远差于 FCFS (Slide 17 必考反例)**：10 个各需 100 s 的作业同时在 0 到达，$q = 1$ s，假设零切换开销。FCFS 完成时间为 $100, 200, \ldots, 1000$（平均 **550s**）；RR 在 $991, 992, \ldots, 1000$ 才完成（平均 **995.5s**）！

3. SJF / SRTF（最短剩余时间优先）与磁盘利用率算例

- **最优性**：SJF 在非抢占中最优，**SRTF（新到达任务时间 < 当前剩余时间则立即抢占）在所有调度算法中使平均完成/响应时间全局最小**！缺点：长任务**饥饿 (Starvation)**、不公平、需预知未来（可用历史突发**指数平均预测** $\tau_n = \alpha t_{n-1} + (1-\alpha)\tau_{n-1}, 0 < \alpha < 1$）。
- **Lec11 Slide 27–29 磁盘利用率算例（必背！）**：A、B 纯 CPU 密集（跑一周），C 循环执行"**1 ms CPU + 9 ms 磁盘 I/O**"：
  - **FCFS**：A 或 B 独占 CPU 两周，磁盘利用率 $\approx 0\%$；
  - **RR ($q = 100$ ms)**：C 跑 1 ms 后需等 A(100 ms)+B(100 ms)，每 201 ms 磁盘只工作 9 ms，磁盘利用率 $= \frac{9}{201} \approx 4.5\%$；
  - **RR ($q = 1$ ms)**：磁盘利用率达 $\approx 90\%$，但 A/B 间产生海量无意义唤醒与切换开销；
  - **SRTF**：C 每次 I/O 完就绪仅需 1 ms，立即抢占 A/B，磁盘利用率 90% 且 CPU 利用率 100%、切换最少！

7. MLFQ、Linux O(1)、公平共享与 EEVDF (L12 & Sample Q4)

- **多级反馈队列 (MLFQ) 与 Sample Q4 满分详解【16 分原题！】**：
- **规则**：新作业进最高级 Q_0；在当前级**用完整个时间片则降一级**；在时间片用完前主动让出 CPU（I/O 阻塞）则**留在原级**；高优先级队列抢占低优先级。
- **欺骗攻击与防御 (Slide 12)**：黑白棋程序 (Othello) 故意在时间片快用完前插入无意义的 `printf` 短 I/O，从而一直留在最高优先级队列！**防御**：按在该级**累计使用的总 CPU 时间**计配额（用满即降级），并定期做全局 Priority Boost 防长作业饥饿。
- **【Sample Q4 原题追踪】**：$Q_0(q=2), Q_1(q=3), Q_2(\text{FCFS})$，无 Boost。
  - **A** (时刻 0 到，共需 6 CPU，不阻塞)；**B** (时刻 1 到，共需 3 CPU，不阻塞)；**C**（时刻 2 到：`1 CPU -> 2 I/O -> 1 CPU -> 2 I/O -> 1 CPU` 结束，I/O 完回原队列）。

Time	Job	Queue	关键状态转移事件
0–1	A	Q0	t=0 A 入 Q0 运行；t=1 B 到达入 Q0 队尾
1–2	A	Q0	t=2 **A 用满 Q0 配额 2 降入 Q1**（剩 4）；C 到达入 Q0（排 B 后）
2–3	B	Q0	Q0 中 B 排在 C 前，B 开始运行
3–4	B	Q0	t=4 **B 用满 Q0 配额 2 降入 Q1**（剩 1，排 A 后）
4–5	C	Q0	t=5 C 跑完 1 单位主动阻塞 I/O（$[5,7)$），未用满配额**保持 Q0**
5–6	A	Q1	Q0 空，运行 Q1 队首 A（在 Q1 已跑 1/3）
6–7	A	Q1	A 在 Q1 已跑 2/3；**t=7 C 完成 I/O 返回 Q0**，抢占 A！
7–8	C	Q0	t=8 C 跑完第 2 单位再次阻塞 I/O（$[8,10)$），保持 Q0
8–9	A	Q1	Q0 空，A 跑完在 Q1 的第 3 单位；**t=9 A 用满 Q1 配额 3 降入 Q2**（剩 1）
9–10	B	Q1	运行 Q1 中的 B；**t=10 B 全部完成 (Finish=10)**；**t=10 C 从 I/O 返回 Q0**
10–11	C	Q0	C 跑最后 1 单位；**t=11 C 全部完成（Finish=11）**
11–12	A	Q2	Q0/Q1 均空，运行 Q2 的 A；**t=12 A 全部完成（Finish=12）**

  - **Q4 必答汇总**：降级时刻 = **t=2 (A), t=4 (B), t=9 (A)**；C 从 I/O 返回时刻 = **t=7, t=10**；完成时刻 = **B: 10, C: 11, A: 12**；等待时间：$\mathbf{A} = 12 - 0 - 6 = \mathbf{6}$；$\mathbf{B} = 10 - 1 - 3 = \mathbf{6}$；$\mathbf{C} = (11-2) - 3(\text{CPU}) - 4(\text{I/O}) = \mathbf{2}$。
- **Linux O(1) 调度器 (Slide 13–15)**：140 个优先级队列（140 位位图 $O(1)$ 查找）：**0–99 为实时/内核任务，100–139 为用户态任务（nice）；数值越小优先级越高**！维护 **Active** 与 **Expired** 两套队列数组，Active 任务用完时间片移入 Expired，Active 空时 $O(1)$ 交换两指针。根据 `sleep_avg = sleep_time - run_time` 与交互信用（IC）对用户优先级做 ± 5 奖惩。
- **Max-Min Fairness（最大最小公平，Slides 19–35, 43）**：
- **无权重注水算法**：每轮算均分份额 $f = \frac{C}{\lvert S \rvert}$；若存在需求 $D[i] \le f$，则满足 $A[i] = D[i]$，从 C 中扣除 $D[i]$ 并移出 S，重复直至所有剩余作业 $D[i] > f$。
  - **课件算例**：$C = 10, D = [8, 6] \Rightarrow [5, 5]$；$D = [8, 4] \Rightarrow [6, 4]$；$\boldsymbol{D} = [\mathbf{8, 6, 2}]$：第 1 轮 $f = \frac{10}{3}$，满足 $A_3 = 2$ 剩 $C = 8$；第 2 轮 $f = \frac{8}{2} = 4 \Rightarrow [\mathbf{4, 4, 2}]$。
- **加权 Max-Min 公平 (Slide 43)**：求解归一化公平速率 f 使 $\sum_i \min(r_i, f \cdot w_i) = C$。
  - **课件算例**：$C = 10, r = [8, 6, 2], w = [3, 1, 1] \Rightarrow$ 取 $f = 2$，分配 $\min(8, 6) = 6, \min(6, 2) = 2, \min(2, 2) = 2 \Rightarrow [\mathbf{6, 2, 2}]$。
- **Lottery、WFQ 虚拟时间与 EEVDF (Slides 37–63)**：
- **Lottery Scheduling**：票数 N_i 代表权重，总票数 $N_{\text{ticket}} = \sum N_i$，随机投飞镖 $d \in [1, N_{\text{ticket}}]$，选首个前缀和 $\sum_{k=1}^{j} N_k \ge d$ 的任务。短作业 10 票、长作业 1 票时：1 短 1 长为 $91\frac{9}{9}\%$；10 短 1 长为 $9.9\frac{\%}{0.99}\%$；1 短 10 长为 $50\frac{\%}{9}\%$。
- **WFQ 与系统虚拟时间 $V(t)$ (Slide 50–52)**：流体模型 GPS 逐比特按权重服务；WFQ 每次选在 GPS 中最早完成的包。为避免新流到达时重算完成时间，定义系统虚拟时间增长率与第 $k+1$ 个包的**虚拟完成时间 F_i^{k+1}**：

$$\frac{\partial V(t)}{\partial t} = \frac{C}{N(t)} \quad (N(t) = \text{活跃流总权重}), \qquad F_i^{k+1} = \max\left(V\left(a_i^{k+1}\right), F_i^k\right) + \frac{L_i^{k+1}}{w_i}$$

- **EEVDF (Earliest Eligible Virtual Deadline First, Slide 54–63)**：WFQ 会让高权重流连续透支未来份额，导致最坏服务滞后（Lag = 理想服务 − 实际服务）达 $O(n)q$（其余流被饿 n 个时间片）。**EEVDF 规定：仅调度虚拟开始时间 $S_i^k \le V(t)$（即 $\text{lag} \ge 0$）的合格 (Eligible) 任务中虚拟截止期 F_i^k 最早者**，将最坏 Lag 降至 $\boldsymbol{O(1)q}$（**如 ≤ 0.5，与流数 n 无关**）！

算法 (Slide 63)	截止期偏差	服务滞后 Lag	单次调度复杂度
WFQ / Fair Queueing	q	$O(n) \cdot q$	$O(\log n)$
EEVDF	q	$\boldsymbol{O(1) \cdot q}$	$O(\log n)$
Lottery Scheduling	$O(\sqrt{n}) \cdot q$	$O(\sqrt{n}) \cdot q$	$O(1)$
Round Robin (RR)	q	$O(n) \cdot q$	$O(n)$（判空队列）

8. 实时调度、优先级反转与死锁 (L13 & Sample Q5)

- **实时调度 (EDF) 与选型总表 (Slides 3–14)**：
- **实时 = 可预测性 (Predictability) ≠ 算得快！EDF**（动态优先级，选最近绝对截止期 $D_{\text{abs}} = D_{\text{arr}} + P_i$）可行性充要条件：$\sum_{i=1}^{n} \frac{C_i}{P_i} \le 1$（RMS 静态优先级充分条件为 $\le n\left(2^{\frac{1}{n}} - 1\right) \to \ln 2 \approx 0.693$）。
- **Slide 13 选型表**：CPU 吞吐量 → **FCFS**；平均响应时间 → **SRTF 近似**；**I/O 吞吐量 → SRTF 近似**；CPU 公平 → **RR/WFQ/Lottery/EEVDF**；满足截止期 → **EDF**。应在利用率到达负载曲线**拐点 (Knee)** 时升级硬件（$U \to 100\%$ 时响应时间 $\to \infty$）。
- **优先级反转与火星探路者号 (Slides 15–24)**：低优先级 J_1 持有锁 L，高优先级 J_3 请求 L 阻塞；此时无关的中优先级 J_2 抢占 J_1 持续运行，导致 J_3 被 J_2 间接无限期阻塞（1997 Mars Pathfinder 因此频繁触发看门狗超时重启）。**解法：优先级捐赠/继承 (Priority Donation/Inheritance)** ——J_3 阻塞时临时把自己的高优先级借给持锁者 J_1，直至 J_1 释放锁。
- **死锁四条件、RAG 与检测/避免算法【Sample Q5 12 分原题！】**：
- **Deadlock ⇒ Starvation，但 Starvation ⇏ Deadlock**（饥饿可随高优任务结束而终止，死锁无外部干预永不终止）。
- **死锁四个必要条件 (Coffman)**：**(1) 互斥 (Mutual Exclusion)**；**(2) 持有并等待 (Hold and Wait)**；**(3) 不可抢占 (No Preemption)**；**(4) 循环等待 (Circular Wait)**。
- **资源分配图 (RAG)**：请求边 $P_i \to R_j$，分配边 $R_j \to P_i$。无环 ⇒ 无死锁；**单实例有环 ⇔ 必死锁**【Sample Q5.1–5.2：$P_1 \to R_2 \to P_2 \to R_3 \to P_3 \to R_1 \to P_1$ 成环且单实例，必死锁】；**多实例有环 ⇒ 未必死锁**（若环外进程可跑完归还同类资源实例即可解开）。
- **多实例死锁检测算法 (Deadlock Detection) vs 银行家算法 (Banker's)**：

```
[Avail] = [FreeResources];  UNFINISHED = {all nodes};
do {
    done = true;
    foreach node in UNFINISHED {
        // 死锁检测用: if ([Request_node] <= [Avail])
        // 银行家算法(假装分配后安全性检查)用: if ([Max_node] - [Alloc_node] <= [Avail])
        if (CONDITION) {
            remove node from UNFINISHED;
            [Avail] = [Avail] + [Alloc_node];  done = false;
        }
    }
} until (done); // 死锁检测: UNFINISHED 非空则死锁! 银行家: 全清空则为 Safe State!
```

- **【Sample Q5.4–5.5 原题答案】**：资源总量 $A = 2, B = 1$。已分配：$P_1(1,0), P_2(1,0), P_3(0,1) \Rightarrow$ 当前空闲 `Avail = [0, 0]`。请求：$P_1(0,1), P_2(0,1), P_3(1,0)$。运行检测算法：无任何进程满足 `Request <= [0, 0]`，循环立即结束，`UNFINISHED = {P1, P2, P3}` 非空 ⇒ **系统已死锁！解除方法**：终止（Kill/Abort）其中一个进程（如终止 P_3 回收 $B = 1$，则 P_1, P_2 可依次跑完）或抢占回滚某进程资源。
- **死锁预防四大手段**：(1) 虚拟内存提供看似无限空间（或把换出磁盘视为抢占物理内存）；(2) 不允许等待（拿不到锁立刻释放已持锁重试，易活锁）；(3) 原子一次性拿齐所有锁 `Acquire_both(x,y)` 或外包一层全局锁 z；(4) **最常用：强制全局一致的加锁顺序**（如一律先拿 `x` 再拿 `y`，或网格路由先走东西向 X 再走南北向 Y 维度排序；**注意 Slide 50：`Release` 释放锁的顺序不影响死锁**！）。
- **Safe vs Unsafe vs Deadlocked**：Safe 保证绝不死锁；**Unsafe 当前尚未死锁**，但若后续线程提出最大声明需求则死锁不可避免。现代 OS 对应用层死锁多用**鸵鸟算法（Ostrich / Denial）**忽略。